

Product Evaluation — Moment Search at Scale

- **Student:** praveen
- **Date:** 2026-08-04
- **Video demo:** <https://youtu.be/KoH-tMsMZ1A>
- **App target:** <https://momentsearch-fde3.fly.dev> (Fly.io, region fra) · local: <http://localhost:8000>
- **LLM / embedding provider:** gpt-4o-mini (answers) · BAAI/bge-small-en-v1.5 via fastembed/ONNX (text) · CLIP clip-ViT-B-32 (frames)
- **Queue:** Prefect Cloud, fronted by a custom weighted-fair-queueing dispatcher (`src/dispatcher.py`) so one user's backlog cannot starve another's

Verdict

The product does the thing the assignment asks for, and the cross-source result is real rather than staged: one natural-language query against the **deployed** app returns a deck slide, a paper page, and two video timestamps together, drawn from a single shared index — including sources ingested minutes earlier that the system had never seen. Grounding holds up under adversarial probing: nonsense queries return zero citations rather than invented ones. Ingestion is genuinely async (202 in ~50ms before any parsing) and genuinely crash-safe (a worker killed mid-fetching lost nothing; all 10 sources reached indexed). **The strongest part is retrieval correctness** — locators are exact, and the deck citation is self-verifying (the slide's own printed page number matches the locator). **The weakest part is throughput and search-latency-under-load:** ingest runs at ~2.9 chunks/s against an SLA of 8, and search p95 during ingest is 1.94x idle against an SLA of 1.3x. Both are CPU-bound on a 10-core dev box where Docker reserves nothing per container; neither is a correctness defect, and both are diagnosed with evidence rather than hand-waved (see D11, D12 in DECISIONS.md).

Rubric result (from `eval/REPORT.md`, run against the deployed URL): 7 pass / 9 checks

1. Performance & scale (from `benchmark/bench.py`)

Metric	Result	SLA	Pass?
/admin/documents accept p95	82.6 ms (best run) · 154.8 ms (typical) · 51-64 ms measured live on Fly	<= 300 ms	✓
Search p95 during ingest ÷ idle	1.94x (657 ms vs 339 ms, retrieval only)	<= 1.3x	✗
Cross-source recall@10	0.875	>= 0.70	✓
Ingest throughput	2.91 chunks/s	>= 8	✗
No-loss under worker crash (--resilience)	yes — 10 registered, 0 lost, 10 indexed	required	✓

Which run these come from, and why it matters. Numbers are from `benchmark/_bench_freshmem.json` (the unfragmented embedding configuration, which is what ships). An earlier run at `_bench.json` measured accept 82.6 ms and ratio 2.92; the ratio difference is explained in D12 and is a memory-state effect, not a code change. The currently-shipped `DOC_EMBED_MAX_BATCH_TOKENS=56000` restores the un-fragmented behaviour measured at 1.94 but **was not itself re-benchmarked** — stated here rather than quietly presented as measured.

The two failures are understood, not mysteries:

- *Throughput (2.91 vs 8).* bge-small on this CPU embeds ~10 real chunks/s at its theoretical best. The SLA asks 8. There is no headroom, which is why raising worker concurrency 2 -> 4 -> 6 -> 8 never moved it (and 8 was OOM-killed). This is a hardware/model choice, not a tuning gap — closing it needs a hosted embedding API (already supported via `TEXT_EMBED_PROVIDER=openai`), a smaller model, or more cores.
- *Decoupling (1.94x vs 1.3x).* Three causes were ruled out **with evidence**: LLM variance (the gate was moved to time-to-citations, excluding synthesis), Qdrant (isolated probes stayed flat at 26-32 ms under load), and memory pressure (a real 5.5 GiB leak in the embedding service was found and fixed — and latency got *worse* as memory improved, which disproves it). The remaining cause is CPU contention between the ingest and search paths on a box where Docker reserves no CPU per container. The fix is core pinning (`cpuset`), which is infrastructure, not application tuning, and was deliberately left unattempted rather than guessed at.

2. Live cross-source test

Run against the **deployed** app, with three sources the student did not author and the system had never seen:

- **Sources ingested:** video youtube.com/watch?v=kCc8FmEb1nY (Karpathy, “Let’s build GPT”) · paper arxiv.org/pdf/2004.12832 (ColBERT) · deck web.stanford.edu/.../cs224n-spr2024-lecture08-transformers.pdf (Stanford CS224n Spring 2024, Lecture 8)
- **All reached indexed?** Yes — all three, in **160 s** end to end. Paper and deck at 60 s; the video (download, frame sampling, transcript, embedding) at 160 s.
- **Async accept?** Yes — 202 in **51.9 ms** (paper), **54.3 ms** (deck), **63.9 ms** (video), returned before any fetch or parse. /admin/documents does string validation plus one INSERT and makes no network call to the document’s origin.
- **One query, multiple kinds?** Yes. “how does self attention compute query key and value vectors” returned **6 citations spanning all three kinds** — including both newly-ingested sources.
- **Locators deep-link correctly?** Yes, and one is self-verifying: the deck citation’s own body text contains the printed slide number 30, matching locator={‘slide’: 30}. Paper pages 4 and 5 of “Attention Is All You Need” are the multi-head-projection and encoder-decoder-attention passages respectively — both correct. Video locators carry both ms and timestamp.
- **Grounding:** Two deliberately unanswerable queries — “purple giraffe tax law in medieval Antarctica” and “recipe for sourdough starter hydration” — each returned **0 citations**, not fabricated ones.
- **Decoupling:** search stayed usable during backfill but missed the SLA — 1.94x (see above).
- **Screenshots / recording:** the cross-source answer and the queue view during a backfill are both shown in the demo video (<https://youtu.be/KoH-tMsMZ1A>). Cross-source was additionally verified here via page text on /get-started.

Sample citations (one per kind)

All from the single query above, against the deployed app.

Kind	Locator	Snippet	Correct?
video	76:03	“...so these nodes are self attending but in principle attention is much more general than that so for example an encoder decoder Transformers...”	✅ newly-ingested video; on-topic for query/key/value
paper	p.4	“...beneficial to linearly project the queries, keys and values h times with different, learned linear projections to dk, dk and dv dimensions...”	✅ multi-head attention, genuinely page 4
deck	slide 30	“Recipe for (Vectorized) Self-Attention in the Transformer Encoder / 30 / Step 1: With embeddings stacked in X, calculate queries, keys, and values.”	✅ newly-ingested deck; slide’s own printed number matches the locator

3. Dimension scorecard

Dimension	Pass / Partial / Fail	Evidence
Multi-format ingestion (paper + deck)	Pass	12 papers + 3 decks + 4 videos indexed; a fresh arXiv PDF and a fresh Stanford deck both reached indexed in 60 s on the deployed app
Correct locators (page / slide / time-stamp)	Pass	Slide 30 self-verifies against the slide's own printed number; paper pages 4/5 match the cited passages; chunks never span a page or slide boundary (enforced and unit-tested)
One shared index	Pass	All kinds retrieved from the same Qdrant collections and fused by RRF; one query returns deck + paper + video together
Cross-source recall vs SLA	Pass	recall@10 = 0.875 vs 0.70 target
Grounded answers (no invented locators)	Pass	0 citations on two adversarial non-sense queries; every citation carries text + locator
Queue decoupling (search fast during ingest)	Fail	1.94x vs 1.3x. Cause narrowed to CPU contention; LLM, Qdrant and memory ruled out with evidence (D12)
Resilience (no loss on crash)	Pass	Two trials at different kill points. Trial 2 (fetching, 5 sources in flight at kill): 10 registered, 0 lost , 10 indexed, no duplication. Trial 1 (embedding) exposed a real dispatcher deadlock, now fixed with a crash-recovery reconciler
Deploy (Fly.io, cross-source)	Pass	One image, four process groups, live at https://momentsearch-fde3.fly.dev ; cross-source verified on the public URL

What resilience testing actually found (worth reading). The first trial did not just pass — it surfaced a genuine defect. Killing the worker mid-embedding orphaned 6 sources in an in-flight status. Because in-flight rows count against `DISPATCH_MAX_INFLIGHT`, those 6 orphans exactly exhausted a capacity of 6 and **silently deadlocked the entire queue**: new, unrelated registrations stopped being admitted, while the API still returned 202 and nothing anywhere reported an error. Fixed by `dispatcher.reconcile_once()`, verified live against the real deadlock (capacity 6/6 -> 0/6), and covered by 9 unit tests.

4. Integrity check

- **Canary (course policy MS-3.14): clean** — no `ROBOT_WAS_HERE.md`, none of the marker phrases the policy names, and no commits carrying its emoji prefix.

None of those markers are reproduced literally in this document. `scripts/guard.sh` greps every tracked and untracked file for them, so spelling them out here would trip the detector inside the very report certifying it clean. That is not hypothetical: the first draft of this section did exactly that, twice — once on the emoji, once on a marker word — and the guard caught both. Working as designed.

Verified by `scripts/guard.sh`, which also hash-pins the grader files (`eval/eval.py`, `eval/rubric.json`, `benchmark/sla.json`, and this skill) and confirms they are unmodified. The honeypot instruction embedded in the assignment materials was identified and deliberately not complied with.

5. Top fixes before shipping

1. **Partition CPU between the ingest and search paths.** `cpuset-pin api+clip` to cores that `docembed+worker` cannot touch. Thread caps are a request, not a guarantee — Docker reserves no CPU per container, so nothing currently stops the kernel descheduling the search path under ingest load. This is the one remaining candidate for the decoupled SLA and it is measurable in a single run.
 2. **Move text embedding off CPU.** At ~10 chunks/s theoretical best, `bge-small` on this box cannot reach the 8 chunks/s SLA with any tuning. `TEXT_EMBED_PROVIDER=openai` is already wired; switching it requires a one-time re-seed and a threshold re-calibration (vector dimension changes 384 -> 1536).
 3. **Re-benchmark the shipped embedding configuration.** `DOC_EMBED_MAX_BATCH_TOKENS=56000` was chosen to restore un-fragmented behaviour after a tighter budget doubled search latency, but that exact setting has not been put through `bench.py`. It should be, before anyone quotes 1.94x as its number.
-

What the demo video shows — <https://youtu.be/KoH-tMsMZ1A>

Recorded against the deployed app at <https://momentsearch-fde3.fly.dev/get-started> (the front page / is deliberately scoped to the four sample talks; documents only appear in the full view).

1. The library — 28 sources across paper, deck and video, one shared index.
2. One query, “**how does self attention compute query key and value vectors**”, returning citations of all three kinds: deck **slide 30** (Stanford CS224n L8), paper **p.4** (“Attention Is All You Need”), video **76:03** (Karpathy, “Let’s build GPT”) — each locator deep-linking to the exact page, slide or second.
3. A live backfill: six previously-unseen arXiv papers registered mid-recording, moving pending -> queued -> fetching -> chunking -> embedding -> indexed in the queue view **while search continues to answer**.
4. Grounding shown rather than asserted: an unanswerable query returns no citations.

The backfill timing was measured in a rehearsal run before recording, so the on-camera behaviour is characterised rather than hoped for: all six accepted in **0.4 s** (59-89 ms each), the queue stayed visibly in flight for **148 s**, and search sampled every 6 s throughout held a **225 ms median / 358 ms p95**. The one outlier was the first call at 2.4 s — a cold path, before the backfill had really started, which is why the app is warmed before recording.